- **Complete component coverage**: The model must implement the required subtasks for each digit position i.e. 6 SA, 6 ST, 6 SV, and 7 answer digit finalization nodes.
- **Component-level behavior**: Each node implementing a subtask must obey certain conditions. For instance, a node implementing $ST2$ must:
  - Attend primarily to tokens D_2 and D'_2 (post-softmax attention > 0.01)
  - Be positioned after D'_2 but before the "=" token
  - A Primary Component Analysis (PCA) should show 3 distinct clusters. (See Fig. 7.)
  - Given three test question sets, created specifically to trigger the 3 $ST2$ values, each question set should align with exactly one of the PCA clusters.
- **Algorithmic ordering constraints**: Node locations must respect computational dependencies. Since SV1 = TriAdd(ST1, ST0), a node computing SV1 must be positioned after nodes computing ST1 and ST0. Our 6-digit model has over 30 such ordering constraints.
- **Causal verification through intervention**: When we identify a candidate node for a specific subtask (e.g. ST2), we can construct paired test questions differing only in that subtask's value. By intervening on the candidate node and swapping its activations between the test pairs, we can predict the model's output from nearly 2 million possible answers.

For models with more than 6 digits, the constraints list grows further. The subtraction and mixed algorithms each have a more extensive constraints list. With 49 models to test, we developed an automated testing framework to:

- Systematically search for nodes over each subtask and digit position combination retaining only those that satisfy all the constraints in our algorithm
- Compare the list of nodes found with the algorithmic requirements (including intra-node relationships).
- Visualize the results.

4.2.2 Addition model results

We tested a single 1-layer addition model. Our framework found the SA_n and SC_n (single-digit carry calculated during answer tokens) nodes as described by Quirke and Barez [2024] that give the model its 99% accuracy.

Testing our fifteen 2-layer addition models, we find all models contain the subtasks required by our addition algorithm, and satisfy the algorithmic requirements. This consistent pattern suggests we have identified a robust and generalizable algorithm that naturally arises when transformer models are trained on addition in our setup. There is some variability between models:

- In a given model, ST_N is not as "tidy" as shown in Fig. 1. Different models select different attention heads to calculate say ST2. Redundantly, some models have two nodes that both calculate say ST1. See Tab. 1.
- Some models combine 2 attention heads (with the same token position and layer) to calculate say SA2. Many models only use one attention head.
- For ST nodes, the PCA trigrams have difference appearances in each attention head (- but all have 3 clusters aligned to the ST values. See Fig. 7.
- All models have ST nodes, but some models also have and use SC nodes. The SC nodes are redundant: the ST values are equivalent and the SV values are superior! See Tab. 1.

Table 1: For a sample 5-digit addition model, we show a compacted location map for the SA, SC and ST subtasks. Interestingly 1) the ST nodes are in a semi-random order 2) the second $ST1$ node is redundant 3) the model uses SC nodes which are redundant 4) each SA subtask is shared across two attention heads.

	(P11) D'1	(P12) D'0	(P13) =	(P14) +	(P15) A6	(P16) A5	(P17) A4	(P18) A3	(P19) A2	(P20) A1
L0H0	ST2	ST3	ST1	ST4	SC4	SC3	SC2	SC1	SC0	
L0H1	ST1		ST0		SA5	SA4	SA3	SA2	SA1	SA0
L0H2				ST5						

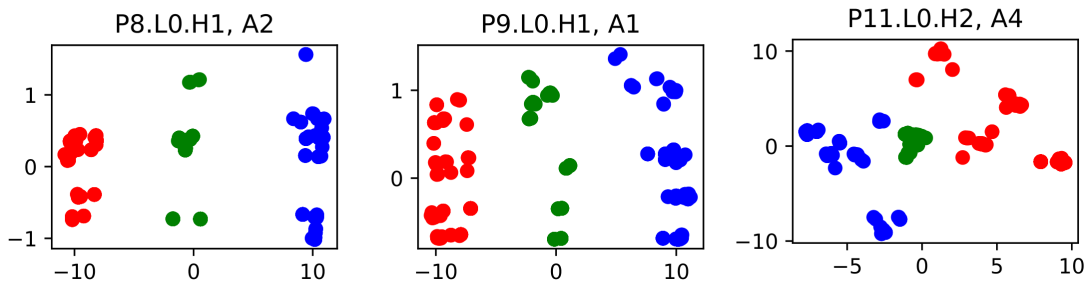

Figure 7: For a 5-digit addition model, these three sample PCAs each show 1 attention head at a token position tested for 1 answer digit. Each dot represents a question, colored by the question's expected ST value (0, 1, or U). While the PCA data varies across cases, the 3 ST classes form visually distinct clusters in each plot, suggesting that the attention head's activation patterns differ systematically based on ST value and that the model could potentially distinguish between these 3 cases.

4.2.3 Mixed model results

Testing our accurate "mixed" (both addition and subtraction) models, we find all models contain the subtasks required by our addition algorithm, and satisfy the algorithmic requirements.

The mixed models have the same types of variability that the addition models have.

Evolution of Polysemantic Nodes. Zhang et al. [2024] identified symmetries between addition and subtraction circuits, with the same attention heads handling both operations. We expand on this finding.

Consider the mixed models that we initialized with accurate, smaller addition models before training. The majority of inherited nodes became polysemantic during training. Rather than developing entirely new circuits for subtraction, the models adapted their addition circuits to handle both operations. For example, a node that originally only performed Base Add (SA) often learns to handle three subtasks: addition (SA), positive-answer subtraction diff (MD), and negative-answer subtraction diff (ND). (Refer to Tab. 2.)

Table 2: For this mixed model, in the last 5 tokens, polysemantic attention heads simultaneously generate outputs for the three question classes (S, M and N). Other heads calculate the question class by attending to the question operation (OPR) token and the answer sign (SGN) token. We assume the MLP layers then select the output appropriate for the class.

	(P9) D'3	(P10) D'2	(P11) D'1	(P12) D'0	(P13) =	(P14) A7	(P15) A6	(P16) A5	(P17) A4	(P18) A3	(P19) A2	(P20) A1
L0H0	MT4			MT3 MT3		ST4 MT4 OPR	SC4	SC3	SC2 NB2	SC1 MB1 NB1	SC0 MB0 NB0	OPR SGN
L0H1	ST4	ST2 MT2	ST1 MT1	ST3	ST0 MT0		SA5 MD5	SA4	SA3	SA2	SA1	SA0
L0H2						ST5 OPR SGN	SA4 ND5	MD4 ND4	MD3 ND3	MD2 ND2	MD1 ND1	MD0 ND0
L0H3							OPR SGN	OPR SGN	OPR SGN	OPR SGN	OPR SGN	OPR SGN

We believe that this adaptation was likely because these three operations share a common structure: each maps a pair of digits (10 possibilities for D_n× 10 possibilities for D'_n= 100 input cases) to a single-digit result (10 possible outputs). Similarly, some models adapted nodes that originally handled carry bits (ST) to also process borrow bit operations (MT).

Table 3: Mixed models re-use most inserted addition-model nodes. Many inserted nodes become polysemantic during training - performing addition, positive-answer subtraction **and** negative-answer subtraction subtasks simultaneously. For a sample mixed model that uses 96 nodes and had 48 nodes inserted, this table shows inserted node reuse.

	Used		**Inserted**	
Question class	**#**	**%**	**#**	**%**
All questions	96		48	
Addition	61	64%	42	88%
Positive-answer subtraction	70	73%	40	83%
Negative-answer subtraction	53	55%	29	60%

5 Conclusion

Our analysis of 49 transformer models demonstrate their ability to learn highly accurate mathematical algorithms, achieving over 99.999% accuracy on addition tasks up to 15 digits. By investigating how models implement computations using attention heads and MLPs, we uncovered novel computational approaches distinct from traditional human calculation methods. Parameter transfer experiments revealed that features originally specialized for addition could be repurposed for subtraction, with many nodes becoming polysemantic.

Our research provides insights into how transformers learn precise algorithmic computations across related tasks, developing a reusable library of mechanistic interpretability tools that advances our understanding of neural network computational learning.

6 Limitations and Future Work

While we identify and test the functional *role* of each component in the mixed model algorithm, we do not analyze the specific *data representations* and *transformations* used by polysemantic nodes, SGN nodes, and OPR nodes in the residual stream, nor the detailed mechanisms by which MLP layers process this information. Although our algorithmic-level analysis may be more generalizable across models, this approach leaves gaps in understanding how the identified components actually encode and manipulate information.

Our automated framework for discovering algorithm subtasks in models, while instrumental in accelerating our research, has limitations. Some aspects are specific to our math models. Some aspects are generic and have already been used in the published paper by Harrasse et al. [2025].

Fully reverse engineering neural networks faces significant challenges, including distributed parallel computations difficult to trace to specific components. We made progress on an alternative approach: a declarative language describing algorithms in terms of subtasks and a framework to test these descriptions against models. However, more work is needed to refine this method and quantity our certainty when comparing evidence gathered from models against an algorithmic description.

Our models may be useful to researchers investigating the universality of representations / circuits across models.

References

Hanseul Cho, Jaeyoung Cha, Pranjal Awasthi, Srinadh Bhojanapalli, Anupam Gupta, and Chulhee Yun. Position coupling: Improving length generalization of arithmetic transformers using task structure, 2024. URL `https://arxiv.org/abs/2405.20671`.

Abir Harrasse, Philip Quirke, Clement Neo, Dhruv Nathawani, Luke Marks, and Amir Abdullah. Tinysql: A progressive text-to-sql dataset for mechanistic interpretability research, 2025. URL `https://arxiv.org/abs/2503.12730`.

Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL `https://transformer-circuits.pub/2025/attribution-graphs/biology.html`.

Tiedong Liu and Bryan Kian Hsiang Low. Goat: Fine-tuned llama outperforms gpt-4 on arithmetic tasks, 2023.